

Ambiente de Verificación para un Bus con Múltiples Dispositivos

David Leiton Flores, dleitonf@estudiantec.cr

Wilbert Roman Rodriguez, alonso05rr01@estudiantec.cr

I. INTRODUCCIÓN

En este documento se presenta el diseño del plan de pruebas para la verificación de un bus de comunicación con múltiples dispositivos ($M = 4$), con arbitraje round-robin distribuido y ancho de palabra configurable en tiempo de compilación (16, 32 o 64 bits), de los cuales los 8 bits más significativos codifican la dirección de destino —o el identificador de *broadcast*— y el resto transporta el dato útil. Se describe el ambiente de verificación desarrollado, sus componentes, las transacciones que circulan entre ellos y las pruebas implementadas. El ambiente fue desarrollado en SystemVerilog siguiendo el modelo estándar de aleatorización controlada por capas visto en clase.

II. TEST PLAN

A. Caso general

Para probar el bus se plantea un caso general en el que se cumple lo siguiente:

- Se genera un número configurable de transacciones (por defecto, 200) repartidas aleatoriamente entre los 4 dispositivos como origen.
- El tipo de cada transacción (destino válido, *broadcast* o destino inexistente) se selecciona con una distribución aproximadamente uniforme entre las tres opciones, de forma que ningún escenario domine la corrida por la simple diferencia en el tamaño de su espacio de valores válidos.
- Las direcciones de destino, el dato transportado, el largo de paquete declarado y el retardo antes de transmitir se generan de forma aleatoria y controlada mediante *constraints*.

B. Casos dirigidos (inyección desde el Test)

A diferencia de un esquema basado en PLUSargs, los casos dirigidos de este ambiente se construyen directamente como objetos `trans_bus` completamente especificados (sin pasar por `randomize()`) dentro de `test.sv`, y se entregan al Agente a través del mailbox `tst_agnt_mbx` antes de arrancar el tráfico aleatorio. El Agente los distingue de las transacciones aleatorias porque los recibe por ese canal, y los procesa primero, en un ambiente todavía libre de tráfico de fondo. Los casos implementados son:

- **Reset a mitad de transacción:** dispositivo 3 solicita el bus hacia el dispositivo 0, y a los pocos ciclos (muy por debajo de lo que tarda una transacción real en

completarse) se dispara un pulso de `reset`, garantizando interrumpirla en curso. Va primero en la cola para no mezclarse con otros casos.

- **Broadcast agresivo:** dispositivo 0 envía un *broadcast* con `retardo=0` y `payload` en todo unos, para exigir al árbitro una respuesta inmediata ante el escenario de mayor *fan-out* posible.
- **Dirección inválida límite:** dispositivo 1 envía hacia `destino=drvrs` (el primer valor fuera de rango posible, el más cercano a la frontera entre válido e inválido).
- **Payload en el otro extremo:** dispositivo 2 envía un paquete válido con `payload=0x0000` y `retardo=0`.
- **Ráfaga:** 8 paquetes consecutivos del dispositivo 0 hacia el dispositivo 1, sin espera entre ellos, cada uno con un `payload` distinto (para que el emparejamiento del scoreboard, basado en el dato transportado, no confunda una entrega con otra).
- **Contención:** los dispositivos 1, 2 y 3 envían, en el mismo instante, un paquete válido hacia el dispositivo 0, forzando al árbitro a resolver una competencia de tres contra uno por el mismo destino.
- **Subflujo forzado (UNDERFLOW):** dado que un DUT correcto nunca concede una transmisión (`pop`) a un dispositivo sin datos pendientes (`pndng=0`), esta condición de protocolo no ocurre de forma natural. Para demostrar que el camino de detección existe y llega hasta el reporte final, el Test inyecta una observación sintética directamente en el mailbox del Checker que representa exactamente esa violación.

C. Capacidades de aleatorización requeridas

El ambiente de pruebas incluye capacidad de aleatorización de:

- **Número de transacciones:** variable pública `num_transacciones` de la clase `agent`, configurada por `test.sv` antes de arrancar el ambiente. Es un total global de la corrida, no un número independiente por terminal; el origen de cada transacción sí se aleatoriza.
- **Largo de los paquetes (16, 32, 64 bits):** campo `pckg_sz` de `trans_bus`, con la constraint `c_pckg_sz { pckg_sz inside {16, 32, 64}; }`. El ancho físico real de la palabra en el bus (parámetro `bits` de `trans_bus/bus_if/DUT`) se fija en tiempo de compilación; se verificó explícitamente mediante tres binarios compilados por separado (16, 32 y 64 bits) que el ambiente entrega e integra correctamente

en los tres casos (ver Sección VIII). Dentro de una misma corrida, `pckg_sz` queda como metadato descriptivo de la transacción y no cambia el ancho físico compilado.

- **Tiempos de envío:** campo `retardo` de `trans_bus`, con la constraint `c_retardo` que lo limita al rango (0, 20) ciclos de reloj antes de que el Driver levante `pndng`.
- **Mensajes con errores:** campo `destino` de `trans_bus` cuando `tipo == INVALIDA`, restringido por `c_destino` al rango `[drvrs, 254]` — direcciones que no corresponden a ningún dispositivo real del sistema.
- **Direcciones de destino:** campo `destino` cuando `tipo == VALIDA`, restringido por `c_destino` al rango `[0, drvrs-1]`, excluyendo además el propio origen.
- **Identificador de broadcast:** campo `destino` fijado en 255 (8'hFF) cuando `tipo == BROADCAST`, también en `c_destino`. La frecuencia relativa de los tres tipos (`VALIDA/BROADCAST/INVALIDA`) se controla aparte con `c_tipo_dist`, una constraint de distribución (`dist`) que evita que el tamaño desigual de cada rango de destino sesgue el muestreo hacia `INVALIDA`.

D. Casos de esquina

Se identificaron e implementaron los siguientes casos de esquina:

- **Reset con una transacción en curso:** descrito arriba; resultado documentado en la Sección VIII.
- **Máxima concurrencia en el árbitro:** tres dispositivos distintos solicitando el mismo destino en el mismo instante (caso de contención descrito arriba).
- **Ráfaga sostenida desde un mismo origen:** 8 transacciones consecutivas sin espera entre ellas hacia el mismo destino, para observar el comportamiento del árbitro ante una cola que nunca se vacía.
- **Broadcast sin espera:** un *broadcast* con `retardo=0`, el escenario de mayor *fan-out* combinado con la menor latencia de solicitud posible.
- **Frontera de direcciones inválidas:** un mensaje dirigido exactamente a `destino=drvrs`, el primer valor fuera de rango, en vez de una dirección inválida arbitraria.
- **Extremos del espacio de datos:** un payload en todo unos y otro en todo ceros, como valores límite del ancho de dato.
- **Violación de protocolo (UNDERFLOW):** descrito en la sección anterior, inyectado sintéticamente porque el DUT correcto nunca lo produce por sí mismo.
- **Rotación completa del árbitro (wrap-around):** cubierto mediante cobertura funcional en vez de un caso dirigido puntual — ver Sección II-E.

E. Cobertura funcional del arbitraje

Además de los casos anteriores, el Checker mantiene un `covergroup` (`cg_arbitraje`) que cruza el dispositivo que recibe el turno actual contra el que lo tuvo inmediatamente antes, sobre cada evento de envío observado. Esto permite confirmar, sin necesidad de un estímulo dirigido puntual, que

el árbitro rota realmente entre todos los dispositivos —incluida la transición de `drvrs-1` de vuelta a 0— durante el tráfico general de la corrida. Al correr por medio del `covergroup`, tenemos lo siguiente: [TEST] Cobertura arbitraje: 85.4% en la consola.

III. ESTRUCTURA DEL AMBIENTE

El ambiente está compuesto por los bloques *Test*, *Environment* (que instancia y coordina al resto), *Agente*, *Driver* (manejador padre-hijos), *Monitor* (observador padre-hijos), *Checker* y *Scoreboard*, además de la interfaz `bus_if` hacia el DUT.

A continuación se detallan los paquetes que circulan entre los bloques:

- **Mailbox `tst_agnt_mbx`** (`trans_bus`): *Test* → *Agente*. Transporta los casos dirigidos, como objetos `trans_bus` ya completamente especificados (ver Sección IV-A).
- **Mailbox `agnt_drv_mbx`** (`trans_bus`): *Agente* → *Driver*. El mismo objeto `trans_bus` (dirigido o aleatorio) que el *Agente* reparte también hacia el *Scoreboard* por un canal aparte.
- **Mailbox `agnt_sb_mbx`** (`trans_bus`): *Agente* → *Scoreboard*. Copia de la “intención” de cada transacción, usada por el *Scoreboard* para saber qué debe esperar recibir.
- **Mailbox `mon_chkr_mbx`** (`mon_obs`): *Monitor* → *Checker*. Cada observación cruda incluye un campo evento que distingue si el Hijo capturó un envío (`EVT_ENVIO`, sobre `pop/D_pop`) o una llegada real (`EVT_LLEGADA`, sobre `push/D_push`).
- **Mailbox `chkr_sb_mbx`** (`trans_sb`): *Checker* → *Scoreboard*. El veredicto de cada evento observado.

Todos los tipos anteriores (`trans_bus`, `trans_sb`) están parametrizados por `bits` y `drvrs`, y cada mailbox que los transporta declara explícitamente esos parámetros (`mailbox # (trans_bus # (bits, drvrs))`), de forma que el ancho de payload compilado se propaga de manera consistente por todo el ambiente — ver la nota sobre este punto en la Sección IX.

IV. OBJETOS PARA TRANSACCIONES

A. `trans_bus`

Este objeto representa una transacción individual sobre el bus, parametrizado por `bits` (ancho del payload, 16 por defecto) y `drvrs` (número de dispositivos, 4). Sus atributos son:

- `origen`: dispositivo que genera la transacción.
- `destino`: dispositivo receptor, o 255 para *broadcast*.
- `payload`: dato de `bits` asociado a la transacción.
- `pckg_sz`: largo de paquete declarado (16, 32 o 64).
- `tipo`: `enum tipo_pkt_e {VALIDA, BROADCAST, INVALIDA}`.
- `retardo`: ciclos de reloj a esperar antes de que el Driver levante `pndng` por esta transacción.
- `t_envio`: instante de simulación en que el Driver efectivamente presenta el dato en `D_pop`. Se estampa

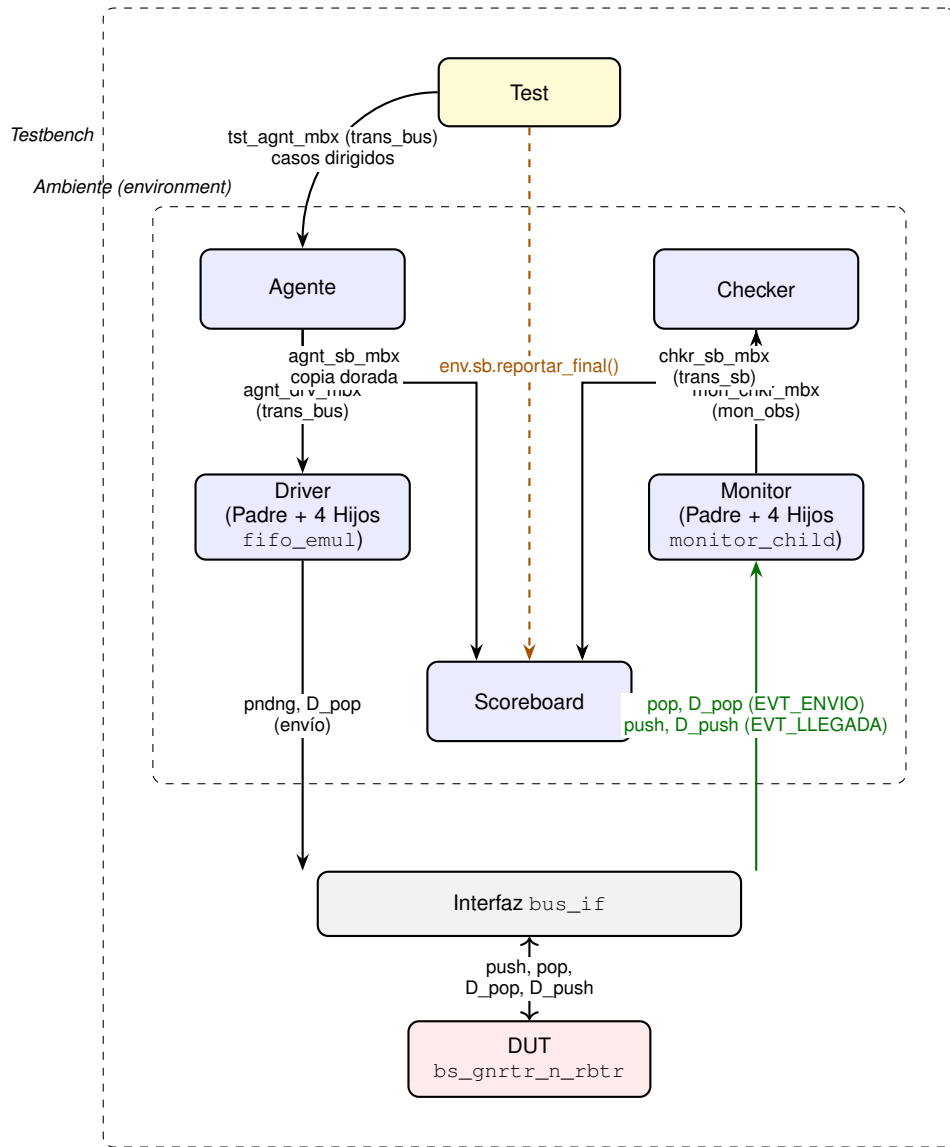


Fig. 1: Estructura del ambiente de verificación: bloques, mailboxes y dirección de las flechas de comunicación. El Monitor observa ambos lados del protocolo del DUT: envío (pop/D_pop) y llegada (push/D_push).

en el Driver, no en el Agente, precisamente porque el retardo y la espera del árbitro hacen que ambos instantes difieran.

Las constraints definidas son `c_origen`, `c_tipo_dist` (distribución pareja entre los tres tipos), `c_destino` (rango de destino según tipo) y `c_pckg_sz` (largo de paquete).

B. `mon_obs`

Representa una observación cruda del Monitor sobre las señales físicas del bus. Sus atributos son `bc_id` (Hijo que la capturó), `evento` (`EVT_ENVIO` o `EVT_LLEGADA`), `raw_data` (el dato tal como salió de `D_pop` o `D_push`, según el evento), `dest` (para `EVT_ENVIO`, decodificado del byte alto de `raw_data`; para `EVT_LLEGADA`, igual a `bc_id`, porque el propio Hijo que observó el evento es el

dispositivo destino) y `pndng` (estado de la señal en el mismo instante, solo relevante para `EVT_ENVIO`).

C. `trans_sb`

Representa el veredicto que el Checker envía al Scoreboard. Contiene el dato observado, el destino, el resultado (`enum resultado_e {COMPLETADO, PERDIDO, UNDERFLOW}`, definido en un archivo de definiciones compartido) y el instante de llegada `t_recibido`. El método `cerrar_con_envio(t_envio)` lo completa con el instante de transmisión real (tomado del `trans_bus` original que el Scoreboard emparejó) y calcula la latencia como `t_recibido - t_envio`.

V. COMPONENTES DEL AMBIENTE

A. Dispositivo bajo prueba (DUT)

El DUT es un controlador de bus con arbitraje, cuyo prototipo es:

```

1 bs_gnrtr_n_rbtr #( parameter bits = 1,
2                     parameter drvrs = 4,
3                     parameter pckg_sz = 16,
4                     parameter broadcast = {8{1'b1}}) (
5     input clk,
6     input reset,
7     input pndng[bits-1:0][drvrs-1:0],
8     output push[bits-1:0][drvrs-1:0],
9     output pop[bits-1:0][drvrs-1:0],
10    input [pckg_sz-1:0] D_pop[bits-1:0][drvrs-1:0],
11    output [pckg_sz-1:0] D_push[bits-1:0][drvrs-1:0]
12 );

```

Listing 1: Prototipo del DUT (módulo de arbitraje del bus).

El protocolo observado tiene dos mitades independientes. En el lado de **envío**, el dispositivo levanta `pndng` para anunciar que tiene datos pendientes; el árbitro concede el turno levantando `pop`, y el dispositivo responde colocando el dato en `D_pop`, con la dirección de destino (o `0xFF` para *broadcast*) embebida en el byte más significativo, ya que el DUT no expone un puerto de dirección separado. En el lado de **llegada**, el DUT entrega el dato al destino levantando `push` y colocando el dato en `D_push`; para un *broadcast*, se observó que los `drvrs-1` dispositivos distintos del emisor reciben en el mismo instante de simulación.

Revisando el código fuente real del DUT (`Library.sv`) se confirma que `bs_gnrtr_n_rbtr` no usa un árbitro centralizado —ese esquema existe en el archivo como el módulo separado `bs_gnrtr`, no instanciado en este proyecto— sino un arbitraje distribuido por paso de testigo: cada dispositivo mantiene su propio contador round-robin (`Counter_arb`), sincronizado entre los cuatro porque todos escuchan la misma señal compartida `trn_chng`. Un dispositivo recibe el turno cuando su contador coincide con su propio identificador (`cnt_rbtr == ntrfs_id`); si en ese momento no tiene nada pendiente (`bs_rqst=0`), el turno avanza de inmediato al siguiente dispositivo, sin desperdiciar ciclos en dispositivos inactivos. La condición de recepción (`ntrfs_cntrl_n_rbtr`) compara los 8 bits más significativos del dato recibido contra el propio identificador o contra el patrón de *broadcast* (todos en 1, `8'hFF` por defecto), confirmando el esquema de direccionamiento embebido asumido por este ambiente. También se confirma que `pop` solo puede activarse después de que la máquina de estados de escritura observe `pndng` en alto: un DUT correcto, por diseño, nunca genera la condición de `UNDERFLOW` por sí mismo, lo cual respalda la decisión de inyectarla de forma sintética para ejercitar ese camino del Checker.

Además, el bus físico compartido (`bus`) es de un único bit: cada interfaz (`bs_ntrfs_n_rbtr`) serializa la palabra completa de `pckg_sz` bits antes de transmitirla, y la deserializa al recibirla, mediante los módulos `serializer`. Esto explica la latencia observada en los resultados: al estar dominada por un tiempo de serialización bit a bit proporcional al ancho de palabra, se mantiene estable dentro de cada configuración de `pckg_sz` y escala de forma consistente

entre configuraciones (ver Sección VIII), en vez de crecer con la congestión acumulada.

B. Test-bench

El test-bench (`testbench.sv`) es el módulo superior de la simulación: genera el reloj del sistema con un período de 10 ns (`forever #5 clk = ~clk;`), instancia el DUT y la interfaz `bus_if` —ambos parametrizados con el mismo `PCKG_SZ/DRVRS`, para que el ancho de palabra sea consistente entre la interfaz y el DUT—, los conecta entre sí, y maneja explícitamente la señal de `reset` de la interfaz antes de crear el objeto `test` y llamar a su método `run()`.

A diferencia de un tiempo límite fijo, la simulación usa un mecanismo de cierre dinámico: `test.sv` espera, mediante un event disparado por el Agente, a que toda la generación de estímulos (dirigidos y aleatorios) haya concluido, y luego espera a que el Scoreboard reporte cero transacciones pendientes (con un tope de seguridad, para no colgar la simulación ante un eventual problema real de *starvation* en el árbitro) antes de generar el reporte final y terminar. El tope de seguridad se dimensionó para acomodar el peor caso de serialización con `pckg_sz=64`, el más lento de los tres anchos soportados.

C. Test

El componente de más alto nivel. Configura `num_transacciones` del Agente, construye y envía los casos dirigidos descritos en el Test Plan a través de `tst_agnt_mbx`, arranca el ambiente, espera su finalización mediante el mecanismo descrito arriba, y solicita al Scoreboard la generación del reporte CSV antes de terminar la simulación.

D. Ambiente (*environment*)

Contenedor de todos los componentes. Instancia y conecta Agente, Driver, Monitor, Checker y Scoreboard, así como todos los mailboxes de comunicación entre ellos, y expone sus referencias (`env.agnt`, `env.sb`, `env.chk`) para que el Test pueda interactuar con ellos directamente. Su método `run()` lanza en paralelo (`fork / join_none`) los métodos `run()` de todos los componentes.

E. Agente

Recibe los casos dirigidos por `tst_agnt_mbx` (de forma no bloqueante, mediante `try_get()`) y los procesa primero, antes de generar `num_transacciones` transacciones completamente aleatorias. Reparte cada transacción, dirigida o aleatoria, hacia el Driver y hacia el Scoreboard por canales separados. Al terminar toda la generación, dispara el event `fin_generacion` que le indica a `test.sv` que ya es seguro empezar a preguntar si queda algo pendiente por resolver.

F. Driver (Manejador)

Implementa una arquitectura padre-hijo: un controlador recibe las transacciones del Agente y las enruta, mediante un mailbox interno, a *drvrs* procesos hijo (*fifo_emul*), uno por dispositivo, cada uno corriendo en un proceso independiente. Cada Hijo sigue el protocolo real del DUT: levanta *pndng*, espera a que el árbitro conceda *pop* para su identificador, y solo entonces presenta el dato en *D_pop* con la dirección embebida en el byte alto.

G. Monitor

Sigue la misma arquitectura padre-hijo. Cada Hijo observa, en el mismo flanco de reloj, *ambos* lados del protocolo de su dispositivo: *pop/D_pop* (genera una observación *EVT_ENVIO*) y *push/D_push* (genera una observación *EVT_LLEGADA*). Para evitar condiciones de carrera con las señales que el Driver escribe, el Monitor utiliza un *clocking block* propio (*mon_cb*), separado del que usa el Driver, donde todas las señales relevantes están declaradas como entrada. El Padre retransmite, sin modificar, cada observación hacia el Checker.

H. Checker

Separa el procesamiento según el tipo de evento recibido. Un *EVT_ENVIO* solo se reporta al Scoreboard cuando representa una violación real de protocolo (*pndng*=0 en el instante del *pop*, es decir, *UNDERFLOW*); un envío normal no genera ningún mensaje, porque su confirmación real llega del otro lado. Un *EVT_LLEGADA* siempre se reporta como *COMPLETADO*, con destino igual al *bc_id* del Hijo que lo observó. En paralelo, cada *EVT_ENVIO* alimenta el *covergroup* descrito en la Sección II-E.

I. Scoreboard

Mantiene una cola de transacciones pendientes por dispositivo destino. Al recibir una transacción del Agente, la encola según su tipo: las de destino inválido se guardan aparte hasta que el Driver les asigne su *t_envio* real; las de *broadcast* se encolan en las colas de los *drvrs-1* dispositivos distintos del emisor (el emisor nunca recibe su propio *broadcast*). Al recibir una llegada del Checker, busca entre los pendientes de ese destino el que ya fue transmitido (*t_envio* distinto de cero) cuya palabra física esperada coincide con el dato recibido, con desempate por el *t_envio* más antiguo entre los candidatos —necesario porque el orden de llegada al mismo destino no siempre coincide con el orden de generación cuando varios orígenes compiten por él. Al final de la simulación, cualquier transacción que siga pendiente se reporta como *PERDIDO*, y se escribe el CSV completo.

VI. GENERACIÓN DE REPORTES Y ESTADÍSTICAS

A. Reporte de paquetes en formato CSV

El ambiente entrega un reporte en *reporte_paquetes.csv*, con las columnas de la Tabla I.

Una fila real del reporte, para un *broadcast* que llega a sus tres destinos reales en el mismo instante:

TABLE I: Formato del reporte CSV de transacciones.

Columna	Descripción
<i>t_envio</i>	Instante en que el Driver transmitió el paquete
<i>origen</i>	Dispositivo que generó la transacción
<i>destino</i>	Dispositivo receptor
<i>t_recibido</i>	Instante en que se confirmó la llegada
<i>retraso</i>	$t_recibido - t_envio$
<i>resultado</i>	COMPLETADO, PERDIDO o UNDERFLOW

```
1 22565000,1,0,23985000,1420000,COMPLETADO
2 22565000,1,2,23985000,1420000,COMPLETADO
3 22565000,1,3,23985000,1420000,COMPLETADO
```

B. Histograma de tiempos de retardo (GNUplot)

A partir del reporte CSV se genera un histograma de los retardos de los paquetes con *resultado*=*COMPLETADO* usando GNUplot:

```
if (!exists("archivo")) archivo = "reporte_paquetes.csv"
if (!exists("salida")) salida = "histograma_retardos.png"
datos = sprintf("< awk -F',' 'NR>1 && $6==\"COMPLETADO\" {print $5}' %s", archivo)
stats datos using 1 nooutput
if (exists("xmax")) {
    set xrange [0:xmax]
    rango_visible = xmax - STATS_min
} else {
    rango_visible = STATS_max - STATS_min
}
if (!exists("binwidth")) binwidth = rango_visible / 30.0
bin(x,width) = width*floor(x/width)
set boxwidth binwidth*0.9
set style fill solid 0.6
plot datos using (bin($1,binwidth)):(1.0) smooth freq with boxes
```

Listing 2: Script de GNUplot para el histograma de retardos.

Sobre cinco corridas con semillas distintas y *pckg_sz*=16 (1358 entregas en total), la latencia mínima observada fue 220.000, el percentil 50 fue 1.430.000, y el percentil 95 fue 1.550.000. Se observó una cola de valores atípicos (hasta 39.160.000 en el caso más extremo), asociada a transacciones interrumpidas por el caso dirigido de reset a mitad de transacción (Sección VIII-A) más que a congestión progresiva del tráfico general.

La Figura 2 muestra esta distribución para el caso general (*pckg_sz*=16, 5 semillas agregadas).

La Figura 3 corresponde a una corrida con *pckg_sz*=64, donde se aprecia la misma forma de distribución pero desplazada hacia latencias mayores (concentración entre 5,2 y 5,4 millones de unidades de tiempo), consistente con el mayor costo de serialización de una palabra de 64 bits sobre el bus de 1 bit del DUT.

VII. CONFIGURACIÓN PARA CORRIDA

La simulación se compila y corre mediante *make*, con la semilla de aleatorización y el ancho de palabra controlados desde la línea de comandos:

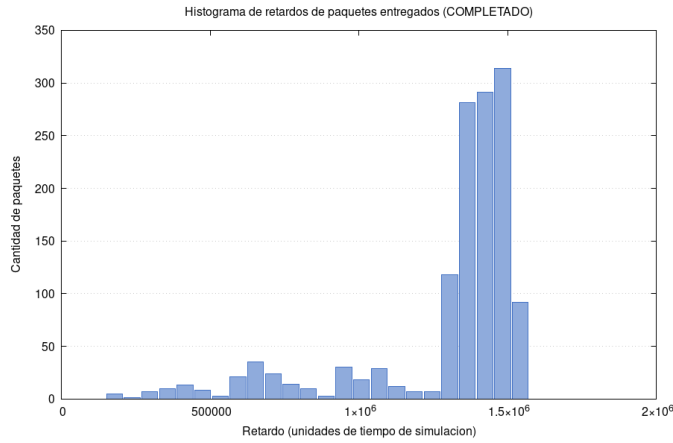


Fig. 2: Histograma de retardos de paquetes entregados, caso general (`pckg_sz=16`, 5 semillas agregadas). El eje X se recorta en 2.000.000 para no comprimir la distribución principal; la cola de valores atípicos asociada al caso de reset se documenta aparte en el texto.

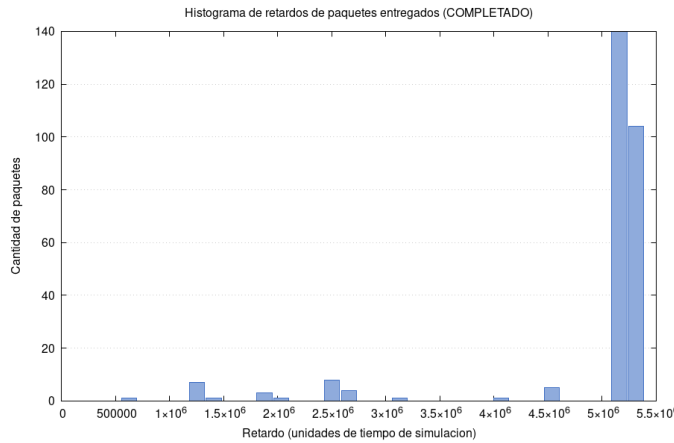


Fig. 3: Histograma de retardos de paquetes entregados, corrida con `pckg_sz=64`.

usada por todo el ambiente. `barrido_pckgsz` compila tres binarios independientes, cada uno con `-pvalue+testbench.PCKG_SZ/BITS` y `+define+PCKG_SZ` fijados al mismo valor, para que el ancho de palabra sea consistente entre el parámetro de clase (que gobierna `trans_bus/scoreboard`) y la macro de preprocesador (que gobierna el ancho de `mon_obs` en el Monitor). De forma análoga, `barrido_drvrs` compila con `-pvalue+testbench.DRVRS` y `+define+DRVRS` fijados al mismo valor, ya que el Monitor dimensiona su arreglo de Hijos (`child[`DRVRS]`) mediante esa misma macro de preprocesador en vez de un parámetro de clase.

VIII. RESULTADOS

Se corrió el ambiente con 5 semillas distintas de regresión (1 a 5), cada una generando 214 transacciones (200 aleatorias más 14 dirigidas), y por separado un barrido de 3 anchos de palabra (16, 32 y 64 bits) con la misma semilla. En las 5 corridas de regresión se verificó, sobre el CSV resultante: ausencia de filas duplicadas, coherencia entre `retraso` y `t_recibido - t_envio` en el 100 % de las 1358 entregas, cero entregas con `origen == destino`, cero *broadcasts* con recepción parcial (los tres destinos reales siempre llegan en el mismo instante), y exactamente una fila `UNDERFLOW` por corrida (la inyectada de forma dirigida, sin duplicados).

A. Hallazgo: reset a mitad de transacción

El caso dirigido de reset con una transacción en curso (dispositivo 3 $\rightarrow$ 0) mostró un resultado consistente en las 5 semillas: en 4 de 5 corridas, la transacción interrumpida por el reset se pierde permanentemente (aparece como `PERDIDO` al cierre de la simulación); en la corrida restante, se recupera, pero con una latencia de 20.960.000 —aproximadamente 15 veces mayor que la latencia típica bajo tráfico normal. Esto sugiere que la recuperación del árbitro tras un reset con solicitudes en tránsito no restaura de forma confiable el turno de un dispositivo que ya tenía `pndng` en alto al momento del reset. Es un hallazgo del comportamiento del DUT ante esta condición específica, no un defecto del ambiente de verificación: el propio Driver del testbench no modifica su estado ante el reset (mantiene `pndng` y el dato en `D_pop` sin cambios), así que el resultado observado depende únicamente de cómo el DUT retoma la arbitración.

B. Verificación del ancho de palabra (16, 32, 64 bits)

Tras corregir la propagación del ancho de payload a través de todos los mailboxes tipados del pipeline (ver Sección IX), el barrido de `pckg_sz` entregó exactamente los mismos resultados de integridad en los tres anchos, sobre la misma semilla:

La tasa de éxito es idéntica en los tres anchos, y la latencia escala de forma aproximadamente proporcional (factor $\sim 1,7$ – $1,8$ por cada duplicación del ancho), consistente con el mecanismo de serialización bit a bit confirmado en el RTL del DUT.

```

1 make compilar # compila una vez (pckg_sz=16 por
  defecto)
2 make simular SEED=7 # corre con una semilla puntual
3 make regresion # corre las semillas 1 a 5 y
4                   # guarda un CSV por cada una en
                   results/
5 make barrido_pckgsz # compila y corre por separado
  con
6                   # pckg_sz = 16, 32 y 64
7 make barrido_drvrs # compila y corre por separado
  con
8                   # drvrs = 2, 4 y 8
9 make grafico # genera el histograma con GNUplot
10 make agregar_regresion # junta las 5 semillas en un
  CSV
11 make grafico_de ARCHIVO=... SALIDA=... # histograma
  a partir de cualquier CSV

```

Listing 3: Targets relevantes del Makefile.

Internamente, `simular` invoca al binario con `+ntb_random_seed=$(SEED)`, la opción de VCS que controla la secuencia de `randomize()`

TABLE II: Resultados del barrido de `pckg_sz`.

	16 bits	32 bits	64 bits
COMPLETADO	276	276	276
PERDIDO	69	69	69
UNDERFLOW	1	1	1
Retardo mínimo	220.000	380.000	700.000
Retardo máximo (típico)	1.550.000	2.830.000	5.390.000

C. Verificación del número de dispositivos (2, 4, 8)

Los casos dirigidos que asumen la presencia de un dispositivo 2 o 3 (payload en el extremo opuesto, contención y el reset a mitad de transacción) se protegieron con una condición `if (drvrs > N)` en `test.sv`, de forma que se omiten automáticamente cuando `drvrs` es demasiado pequeño para que ese dispositivo exista, sin afectar al resto de la corrida. Se verificó explícitamente que, con `drvrs=2`, ninguno de esos tres casos se inyecta.

TABLE III: Resultados del barrido de `drvrs`.

	2 disp.	4 disp.	8 disp.
Filas totales	211	346	606
COMPLETADO	140	276	532
PERDIDO	70	69	73
UNDERFLOW	1	1	1
Duplicados	0	0	0

En ningún caso se observó un origen o destino igual o mayor que el `drvrs` configurado (fuera de los casos BROADCAST e INVALIDA, que usan esos valores a propósito), y el resultado con `drvrs=4` coincidió exactamente con el de las corridas de regresión de la Sección VIII. Con `drvrs=8`, de 64 *broadcasts* generados, 63 entregaron sus 7 copias reales (todos los dispositivos salvo el emisor) en el mismo instante; uno entregó solo 6 de 7, con la séptima copia sin resolver al cierre de la simulación. Dado el mayor volumen de tráfico con más dispositivos (606 filas frente a 346 con `drvrs=4`), este caso es consistente con la limitación de emparejamiento por contenido ya documentada en la Sección IX, y no evidencia un problema adicional.

Fig. 4: Log de inicio: casos dirigidos seguidos del caso general.

Fig. 5: Log de un mensaje hacia una dirección inexistente: transmisión observada en consola (arriba) y fila PERDIDO correspondiente en el reporte final (abajo).

IX. CONCLUSIONES

El ambiente de verificación quedó funcional después de un proceso de depuración iterativo que encontró y corrigió, entre otros:

- La señal de `reset` de la interfaz, inicialmente sin conectar a ninguna fuente real.
- Las direcciones de las señales del *clocking block*, que no correspondían a los puertos reales del DUT.
- El protocolo de arbitraje del Driver, que no esperaba el `pop` antes de transmitir ni embebía la dirección de destino en el dato.
- El Monitor, que solo observaba el lado de envío del protocolo, por lo que un *broadcast* exitoso se reportaba como perdido en 3 de sus 4 destinos.
- El emparejamiento del Scoreboard, que al basarse únicamente en el orden de generación fallaba cuando varios orígenes competían por el mismo destino en un orden distinto al de generación —se reemplazó por un emparejamiento basado en el contenido del dato transmitido.
- Una duplicación en el Checker que generaba dos veces el mismo veredicto de UNDERFLOW por cada violación de protocolo detectada.
- La propagación del ancho de payload (`bits/pckg_sz`) a través de los mailboxes del pipeline y de la propia interfaz `bus_if`, que quedaban fijos en 16 bits por defecto sin importar el parámetro de compilación solicitado —impedía verificar de forma genuina anchos distintos a 16 bits hasta que se corrigió.

Sobre 5 semillas y 1358 entregas de regresión, más un barrido verificado de 3 anchos de palabra, no se encontró ninguna inconsistencia de integridad de datos ni de temporización en el reporte final, y se ejercitaron los tres resultados posibles (COMPLETADO, PERDIDO, UNDERFLOW).

Limitaciones conocidas del ambiente: el número de transacciones se configura como un total global de la corrida, no por terminal; el emparejamiento de llegadas se apoya en que el byte de datos coincida entre el envío y la llegada, lo cual es una heurística razonable —resuelta con desempate por orden de transmisión ante candidatos múltiples— pero no una garantía absoluta ante una colisión exacta de datos entre paquetes en vuelo simultáneamente hacia el mismo destino; y el comportamiento del árbitro ante un reset con

solicitudes pendientes (Sección VIII-A) queda documentado como hallazgo abierto, sin una explicación completa a nivel de máquina de estados del RTL.